UNIDAD 3

Tarea 3.2

ESTUDIANTE: SHEYLA ESTEFANIA MONCADA GUEL
MATRICULA: I24050388
MAESTRO: RUBEN RIOJAS

1. Introducción y Arquitectura Físicas en SGBD

La implementación física de una base de datos relacional representa la etapa en la que el esquema lógico conceptual se traduce a estructuras de almacenamiento reales dentro de un Sistema de Gestión de Bases de Datos (SGBD) como PostgreSQL, MySQL/MariaDB o SQL Server. En esta fase, los nombres de tablas, columnas, tipos de datos físicos y restricciones sintácticas se definen formalmente mediante el Lenguaje de Definición de Datos (LDD o DDL, por sus siglas en inglés).

El objetivo fundamental de este documento es presentar la implementación física completa y validada del Sistema de Información Hospitalaria (HIS) diseñado en las unidades previas. A través del script DDL formal, se garantiza la preservación de la integridad estructural, la coherencia de datos relacionales y el cumplimiento estricto de las reglas de negocio mediante llaves primarias (PRIMARY KEY), llaves foráneas (FOREIGN KEY) con cláusulas de actualización y eliminación en cascada o restricción, y restricciones avanzadas (CHECK, UNIQUE y NOT NULL).

2. Estrategia de Tipos de Datos y Mapeo Físico

Para optimizar el rendimiento y el consumo de almacenamiento en disco, se ha seleccionado un conjunto riguroso de tipos de datos nativos de SQL compatibles con motores de clase empresarial como PostgreSQL y MySQL/MariaDB:

Tipo de Dato Lógico	Tipo de Dato Físico (SQL)	Justificación Técnica y Dominio
Identificadores Únicos (CURP, Cédula)	VARCHAR(18) / VARCHAR(20)	Cadenas de longitud fija o acotada con restricciones sintácticas estrictas mediante CHECK.
Nombres y Descripciones Cortas	VARCHAR(50) / VARCHAR(100)	Cadenas variables optimizadas para almacenamiento sin desperdicio de espacio físico.
Textos Extensos (Diagnóstico, Síntomas)	TEXT	Campos sin límite rígido de caracteres para almacenar resúmenes clínicos detallados.
Fechas y Marcas Temporales	DATE / TIMESTAMP	Registros de precisión cronológica para nacimientos, citas y fechas/horas exactas de atención.
Moneda / Sueldos y Cantidades	DECIMAL(10,2) / INT / SERIAL	Precisión decimal exacta para evitar redondeos numéricos en salarios e identificadores autoincrementables.

3. Reglas de Integridad Referencial y Políticas de Cascadas

El diseño físico establece políticas explícitas de actualización y eliminación de llaves foráneas para mantener la consistencia de los datos clínicos sin generar huérfanos ni perder trazabilidad médica legal:

- **Políticas CASCADE:** Aplicado en tablas dependientes de extensión débil o atributos multivalorados (como TELEFONO_PERSONA, ALERGIA_PACIENTE, PACIENTE_INTERNO, PACIENTE_AMBULATORIO y PRESCRIPCION_DETALLE). Si un registro padre o consulta se elimina, sus detalles asociados se eliminan automáticamente.
- **Políticas RESTRICT:** Aplicado en entidades operativas principales (como MEDICO, PACIENTE, CONSULTA_MEDICA y MEDICAMENTO). Impide la eliminación de un médico o paciente si existen consultas clínicas registradas en el historial, garantizando el cumplimiento de las regulaciones de auditoría hospitalaria.
- **Restricciones CHECK de Dominio:** Garantizan que el sueldo de los médicos sea positivo (sueldo > 0), que el tipo de sangre sea un valor válido del abecedario sanguíneo ('A+', 'A-', 'B+', 'B-', 'AB+', 'AB-', 'O+', 'O-') y que la CURP posea la longitud reglamentaria de 18 caracteres.

4. Script Formal LDD / DDL (.sql) Completo y Validado

A continuación se presenta el código fuente SQL DDL completo, estructurado en el orden de dependencia de entidades (creando primero las tablas independientes y superclases, seguidas de las dependencias y llaves foráneas):

```
-- ================================================================
-- SCRIPT LDD / DDL: SISTEMA DE INFORMACIÓN HOSPITALARIA (HIS)
-- Motor Target: Microsoft SQL Server (T-SQL)
-- ================================================================

-- 0. Selección / Creación de BD
IF NOT EXISTS (SELECT name FROM sys.databases WHERE name = N'hospital_db')
BEGIN
    CREATE DATABASE hospital_db;
END
GO

USE hospital_db;
GO

-- Limpieza previa opcional
DROP TABLE IF EXISTS PRESCRIPCION_DETALLE;
DROP TABLE IF EXISTS MEDICAMENTO;
DROP TABLE IF EXISTS CONSULTA_MEDICA;
DROP TABLE IF EXISTS PACIENTE_AMBULATORIO;
DROP TABLE IF EXISTS PACIENTE_INTERNO;
DROP TABLE IF EXISTS ALERGIA_PACIENTE;
DROP TABLE IF EXISTS PACIENTE;
DROP TABLE IF EXISTS MEDICO;
DROP TABLE IF EXISTS TELEFONO_PERSONA;
DROP TABLE IF EXISTS PERSONA;
GO

-- 1. Tabla PERSONA (Superclase)
CREATE TABLE PERSONA (
    curp VARCHAR(18) NOT NULL,
    primer_nombre VARCHAR(50) NOT NULL,
    ap_paterno VARCHAR(50) NOT NULL,
    ap_materno VARCHAR(50) NULL,
    fecha_nacimiento DATE NOT NULL,
    CONSTRAINT pk_persona PRIMARY KEY (curp),
    CONSTRAINT chk_curp_longitud CHECK (LEN(curp) = 18)
);
GO

-- 2. Tabla TELEFONO_PERSONA
CREATE TABLE TELEFONO_PERSONA (
    id_telefono INT IDENTITY(1,1) NOT NULL,
    curp_persona VARCHAR(18) NOT NULL,
    telefono VARCHAR(15) NOT NULL,
    CONSTRAINT pk_telefono_persona PRIMARY KEY (id_telefono),
    CONSTRAINT fk_telefono_persona FOREIGN KEY (curp_persona)
        REFERENCES PERSONA(curp) ON DELETE CASCADE ON UPDATE CASCADE
);
GO

-- 3. Tabla MEDICO
CREATE TABLE MEDICO (
    cedula_profesional VARCHAR(20) NOT NULL,
    curp VARCHAR(18) NOT NULL,
    especialidad VARCHAR(50) NOT NULL,
    sueldo DECIMAL(10,2) NOT NULL,
    CONSTRAINT pk_medico PRIMARY KEY (cedula_profesional),
```

```
    CONSTRAINT uq_medico_curp UNIQUE (curp),
    CONSTRAINT fk_medico_persona FOREIGN KEY (curp)
        REFERENCES PERSONA(curp) ON DELETE NO ACTION ON UPDATE CASCADE,
    CONSTRAINT chk_medico_sueldo CHECK (sueldo > 0)
);
GO

-- 4. Tabla PACIENTE
CREATE TABLE PACIENTE (
    num_expediente VARCHAR(20) NOT NULL,
    curp VARCHAR(18) NOT NULL,
    tipo_sangre VARCHAR(5) NOT NULL,
    CONSTRAINT pk_paciente PRIMARY KEY (num_expediente),
    CONSTRAINT uq_paciente_curp UNIQUE (curp),
    CONSTRAINT fk_paciente_persona FOREIGN KEY (curp)
        REFERENCES PERSONA(curp) ON DELETE NO ACTION ON UPDATE CASCADE,
    CONSTRAINT chk_paciente_sangre CHECK (tipo_sangre IN ('A+', 'A-', 'B+', 'B-', 'AB+', 'AB-',
'O+', 'O-'))
);
GO

-- 5. Tabla ALERGIA_PACIENTE
CREATE TABLE ALERGIA_PACIENTE (
    id_alergia INT IDENTITY(1,1) NOT NULL,
    num_expediente VARCHAR(20) NOT NULL,
    alergia VARCHAR(50) NOT NULL,
    CONSTRAINT pk_alergia_paciente PRIMARY KEY (id_alergia),
    CONSTRAINT fk_alergia_paciente FOREIGN KEY (num_expediente)
        REFERENCES PACIENTE(num_expediente) ON DELETE CASCADE ON UPDATE CASCADE
);
GO

-- 6. Tabla PACIENTE_INTERNO
CREATE TABLE PACIENTE_INTERNO (
    num_expediente VARCHAR(20) NOT NULL,
    num_cama INT NOT NULL,
    fecha_ingreso DATETIME2 NOT NULL DEFAULT GETDATE(),
    CONSTRAINT pk_paciente_interno PRIMARY KEY (num_expediente),
    CONSTRAINT fk_interno_paciente FOREIGN KEY (num_expediente)
        REFERENCES PACIENTE(num_expediente) ON DELETE CASCADE ON UPDATE CASCADE,
    CONSTRAINT chk_interno_cama CHECK (num_cama > 0)
);
GO

-- 7. Tabla PACIENTE_AMBULATORIO
CREATE TABLE PACIENTE_AMBULATORIO (
    num_expediente VARCHAR(20) NOT NULL,
    fecha_proxima_cita DATE NULL,
    CONSTRAINT pk_paciente_ambulatorio PRIMARY KEY (num_expediente),
    CONSTRAINT fk_ambulatorio_paciente FOREIGN KEY (num_expediente)
        REFERENCES PACIENTE(num_expediente) ON DELETE CASCADE ON UPDATE CASCADE
);
GO

-- 8. Tabla CONSULTA_MEDICA
CREATE TABLE CONSULTA_MEDICA (
    id_consulta VARCHAR(20) NOT NULL,
    cedula_medico VARCHAR(20) NOT NULL,
    num_expediente VARCHAR(20) NOT NULL,
    fecha_hora DATETIME2 NOT NULL DEFAULT GETDATE(),
    diagnostico NVARCHAR(MAX) NOT NULL,
    sintomas NVARCHAR(MAX) NOT NULL,
    CONSTRAINT pk_consulta_medica PRIMARY KEY (id_consulta),
    CONSTRAINT fk_consulta_medico FOREIGN KEY (cedula_medico)
        REFERENCES MEDICO(cedula_profesional) ON DELETE NO ACTION ON UPDATE CASCADE,
    CONSTRAINT fk_consulta_paciente FOREIGN KEY (num_expediente)
```

```
        REFERENCES PACIENTE(num_expediente) ON DELETE NO ACTION ON UPDATE NO ACTION
);
GO

-- 9. Tabla MEDICAMENTO
CREATE TABLE MEDICAMENTO (
    codigo_barras VARCHAR(30) NOT NULL,
    nombre_comercial VARCHAR(100) NOT NULL,
    dosis_recomendada VARCHAR(50) NOT NULL,
    CONSTRAINT pk_medicamento PRIMARY KEY (codigo_barras)
);
GO

-- 10. Tabla PRESCRIPCION_DETALLE
CREATE TABLE PRESCRIPCION_DETALLE (
    id_prescripcion INT IDENTITY(1,1) NOT NULL,
    id_consulta VARCHAR(20) NOT NULL,
    codigo_barras VARCHAR(30) NOT NULL,
    indicaciones NVARCHAR(MAX) NOT NULL,
    cantidad INT NOT NULL,
    CONSTRAINT pk_prescripcion_detalle PRIMARY KEY (id_prescripcion),
    CONSTRAINT fk_prescripcion_consulta FOREIGN KEY (id_consulta)
        REFERENCES CONSULTA_MEDICA(id_consulta) ON DELETE CASCADE ON UPDATE CASCADE,
    CONSTRAINT fk_prescripcion_medicamento FOREIGN KEY (codigo_barras)
        REFERENCES MEDICAMENTO(codigo_barras) ON DELETE NO ACTION ON UPDATE CASCADE,
    CONSTRAINT chk_prescripcion_cantidad CHECK (cantidad > 0)
);
GO
```

5. Pruebas de Validación DML e Integridad de Restricciones

Para comprobar la validez sintáctica del script y el correcto funcionamiento de las restricciones de integridad en el SGBD, se ejecutaron inserciones de prueba (DML) verificando tanto casos de inserción exitosa como violaciones intencionadas de reglas de dominio:

```
-- ================================================================
-- Inserción de Datos de Prueba (DML)
-- ================================================================

-- Médico
INSERT INTO PERSONA (curp, primer_nombre, ap_paterno, ap_materno, fecha_nacimiento)
VALUES ('MECJ850101HDFRRL01', 'Juan', 'Mendoza', 'Cruz', '1985-01-01');

INSERT INTO MEDICO (cedula_profesional, curp, especialidad, sueldo)
VALUES ('MED-1001', 'MECJ850101HDFRRL01', 'Cardiología', 35000.00);

-- Paciente
INSERT INTO PERSONA (curp, primer_nombre, ap_paterno, ap_materno, fecha_nacimiento)
VALUES ('GARA920512MDFRR002', 'Ana', 'García', 'Ríos', '1992-05-12');

INSERT INTO PACIENTE (num_expediente, curp, tipo_sangre)
VALUES ('EXP-2024-001', 'GARA920512MDFRR002', 'O+');

-- Consulta Médica
INSERT INTO CONSULTA_MEDICA (id_consulta, cedula_medico, num_expediente, fecha_hora,
diagnostico, sintomas)
VALUES ('CNS-001', 'MED-1001', 'EXP-2024-001', SYSDATETIME(), 'Hipertensión Arterial', 'Cefalea
y mareo');
GO
```

```
-- ====================================================================
-- B. Prueba de Fallo por Restricción CHECK (Sueldo Negativo)
-- ====================================================================
-- Registramos una persona válida para no violar la FK ni la UNIQUE KEY
INSERT INTO PERSONA (curp, primer_nombre, ap_paterno, ap_materno, fecha_nacimiento)
VALUES ('TEST990101HDFRRL99', 'Prueba', 'Sueldo', 'Check', '1999-01-01');

BEGIN TRY
    -- Intentamos registrar al médico con sueldo negativo
    INSERT INTO MEDICO (cedula_profesional, curp, especialidad, sueldo)
    VALUES ('MED-9999', 'TEST990101HDFRRL99', 'Pediatría', -5000.00);
END TRY
BEGIN CATCH
    PRINT 'Prueba Sueldo Negativo: ' + ERROR_MESSAGE();
END CATCH;
GO


-- ====================================================================
-- C. Prueba de Fallo por Restricción CHECK (Tipo de Sangre Inválido)
-- ====================================================================
-- Registramos otra persona válida para la prueba de paciente
INSERT INTO PERSONA (curp, primer_nombre, ap_paterno, ap_materno, fecha_nacimiento)
VALUES ('TEST880202MDFRR088', 'Prueba', 'Sangre', 'Check', '1988-02-02');

BEGIN TRY
    -- Intentamos registrar un tipo de sangre no admitido ('Z+')
    INSERT INTO PACIENTE (num_expediente, curp, tipo_sangre)
    VALUES ('EXP-999', 'TEST880202MDFRR088', 'Z+');
END TRY
BEGIN CATCH
    PRINT 'Prueba Tipo Sangre Inválido: ' + ERROR_MESSAGE();
END CATCH;
GO
-- Visualización de resultados
SELECT * FROM PERSONA;
SELECT * FROM MEDICO;
SELECT * FROM PACIENTE;
SELECT * FROM CONSULTA_MEDICA;
GO
```

6. Referencias Bibliográficas (Formato APA 7.ª ed.)

- Date, C. J. (2004). An introduction to database systems (8th ed.). Addison-Wesley.
- Elmasri, R., & Navathe, S. B. (2017). Fundamentals of database systems (7th ed.). Pearson.
- PostgreSQL Global Development Group. (2024). PostgreSQL 16.2 documentation: Data definition language (DDL).
- Silberschatz, A., Korth, H. F., & Sudarshan, S. (2020). Database system concepts (7th ed.). McGraw-Hill.